

Universidad de Guadalajara



Centro Universitario de Ciencias Exactas e Ingenierías

Arquitectura de computadoras

Sección: D03

“Actividad 02; Compuertas Lógicas”

Casillas López Derek Gabriel

Código: 221930709

Fecha de entrega 30/Enero/2026

¿Qué es una FPGA? Cómo se relaciona con el lenguaje de descripción de hardware (VERILOG).

Una FPGA "Field-Programmable Gate Array" o arreglo de compuertas programable en campo.

Es un chip reconfigurable: en vez de ejecutar instrucciones como un CPU, tú "armas" dentro del chip un circuito digital a tu medida.

Verilog es un lenguaje de descripción de hardware (HDL).

Con verilog tú describes el comportamiento y/o estructura de un circuito digital, es decir Verilog es el lenguaje con el que describes el circuito que quieres meter en una FPGA.

OBJETIVO

Implementar en Verilog un módulo combinacional que represente las compuertas lógicas fundamentales utilizadas en operaciones de una ALU (AND, OR, NOT, NAND, NOR, XOR y XNOR), verificando su funcionamiento mediante tablas de verdad y simulación, relacionando el diseño con el uso de HDL en dispositivos reconfigurables como las FPGAs.

1. Investigar el concepto de ALU y su relación directa con las operaciones lógicas básicas dentro de un procesador.
2. Investigar qué es una FPGA y explicar cómo se relaciona con Verilog dentro de un flujo típico de diseño digital (descripción → simulación → síntesis/implementación).
3. Describir la función de cada compuerta lógica solicitada mediante su tabla de verdad, su simbología y una explicación breve de su implementación a nivel de transistores (CMOS), usando como evidencia los apuntes realizados.
4. Desarrollar un solo módulo en Verilog que incluya las 7 compuertas y produzca una salida por cada operación lógica.
5. Comprobar que las salidas del módulo coinciden con la teoría, comparando los resultados con las tablas de verdad para todas las combinaciones posibles de entradas.

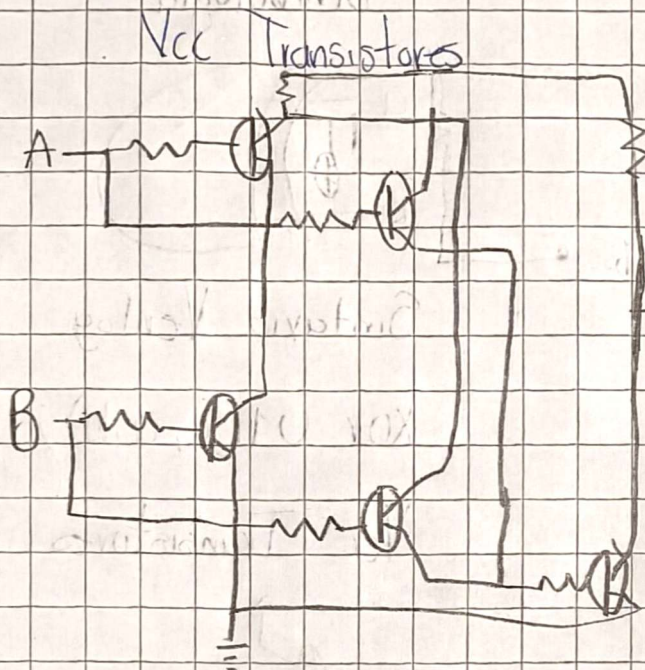
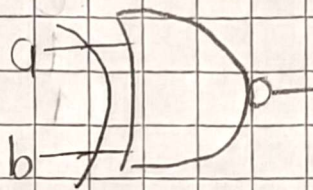
XNOR

a	b	XNOR
0	0	1
0	1	0
1	0	0
1	1	1

Syntax Verilog

```
assign c = ~(a^b);
```

Symbolic

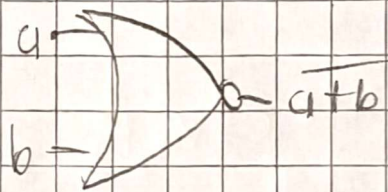


Gated

NOR

a	b	NOR
0	0	1
0	1	0
1	0	0
1	1	0

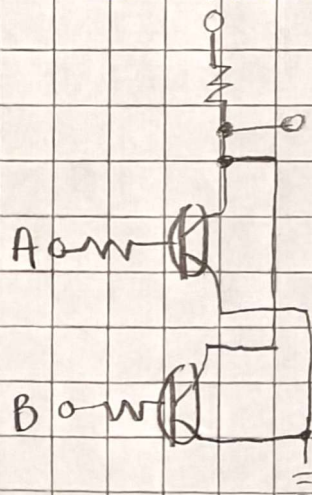
Simbologia



Sintaxis Verilog

```
nor g1(a,b) 1(c,a,b);
```

Vcc Transistores

**XOR**

a	b	XOR
0	0	0
0	1	1
1	0	1
1	1	0

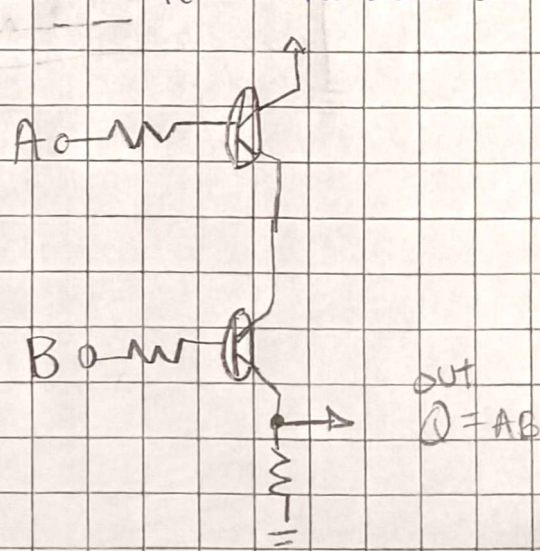
Simbologia



Sintaxis Verilog

```
xor u1(a,b) 1(c,a,b);
```

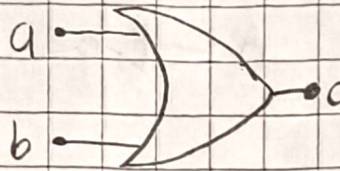
Vcc Transistores



OR

a	b	c
0	0	0
0	1	1
1	0	1
1	1	1

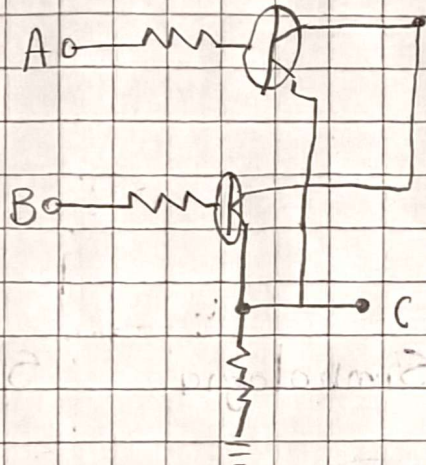
Simbología



Sintaxis Verilog

```
c = a | b;
```

Vcc Transistores



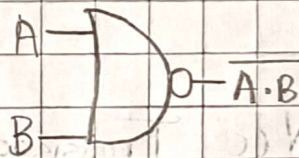
Simbología

Sintaxis Verilog

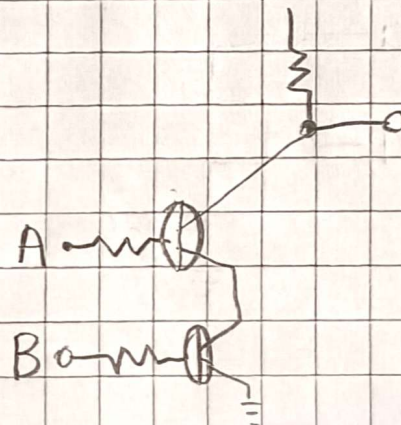
```
nand x1(c, a, b);
```

NAND

a	b	$a \cdot b$
0	0	1
0	1	1
1	0	1
1	1	0



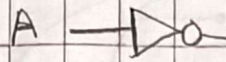
Vcc Transistores



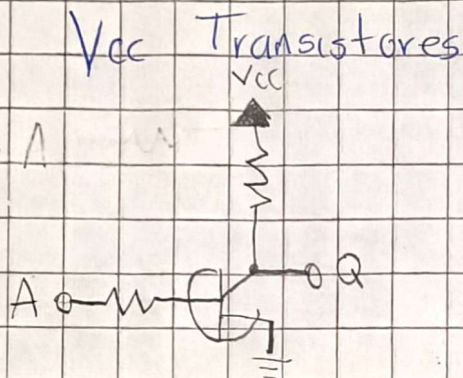
NOT (Inversor)

a	c
0	1
1	0

Simbología

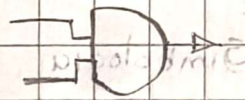


Sintaxis Verilog

 $Y = \sim A;$ **AND**

a	b	c
0	0	0
0	1	0
1	0	0
1	1	1

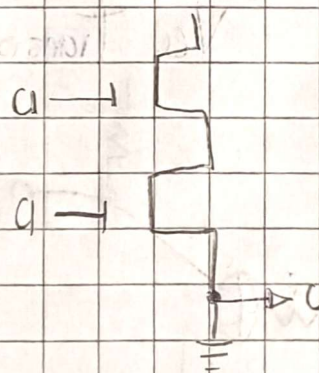
Simbología



Sintaxis Verilog

 $\&$

Vcc Transistores



Código:

```
module compuertas_logicas(
```

```
    input a,
```

```
    input b,
```

```
    output out_and,
```

```
    output out_or,
```

```
    output out_not_a,
```

```
    output out_nand,
```

```
    output out_nor,
```

```
    output out_xor,
```

```
    output out_xnor
```

```
);
```

```
    assign out_and = a & b;    // and
```

```
    assign out_or = a | b;    // or
```

```
    assign out_not_a = ~a;    // not
```

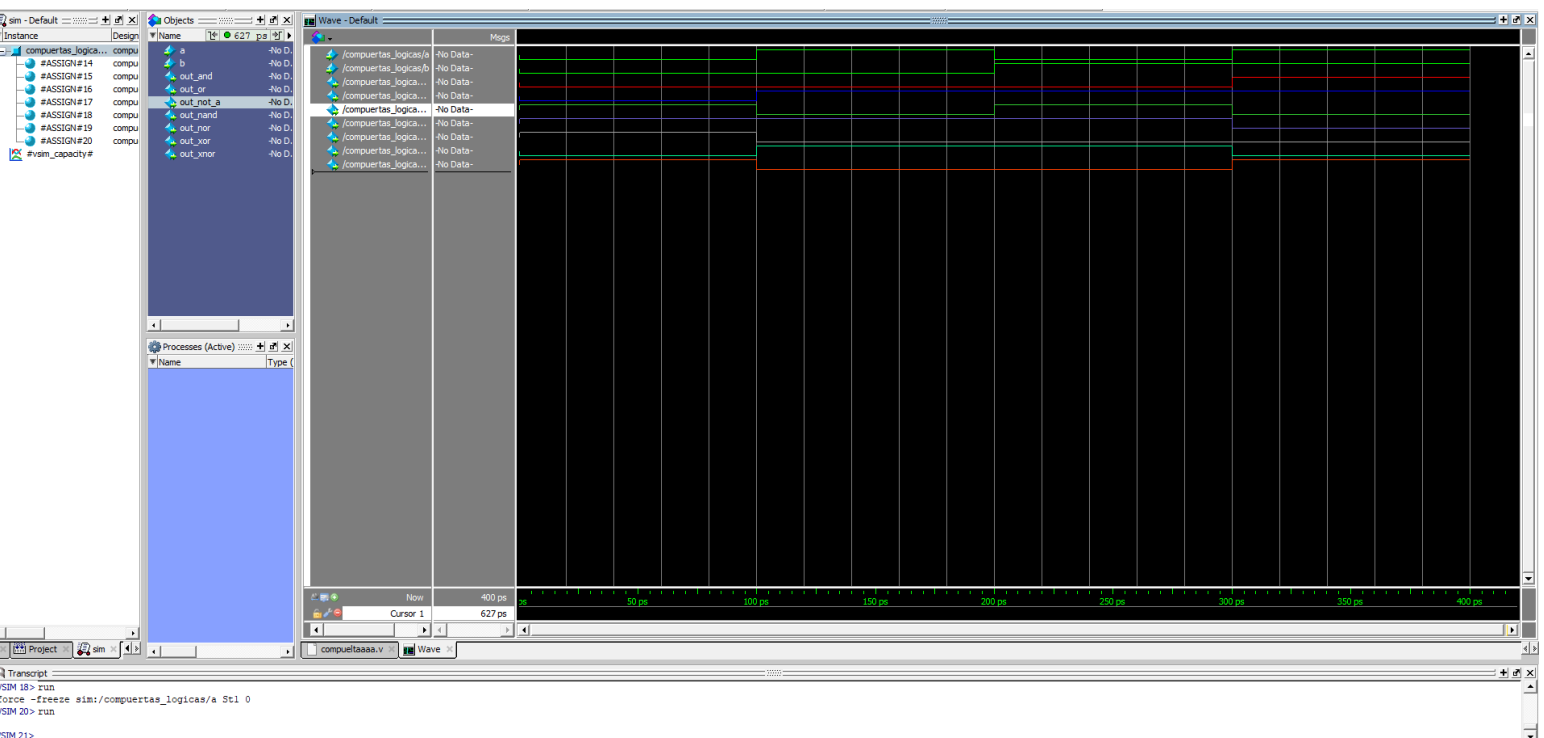
```
    assign out_nand = ~(a & b); // nand
```

```
    assign out_nor = ~(a | b); // nor
```

```
    assign out_xor = a ^ b;    // xor
```

```
    assign out_xnor = ~(a ^ b); // xnor
```

```
endmodule
```



DESARROLLO

Para cumplir los requerimientos, primero se realizó la investigación teórica sobre la ALU, las compuertas lógicas y la relación entre FPGA y Verilog. En la parte lógica, se revisó la definición de cada compuerta y su tabla de verdad; y en la parte física se revisó de manera general la implementación en CMOS, donde el comportamiento lógico se obtiene por redes de transistores que determinan si la salida se conecta a '1' (VDD) o '0' (GND). Como evidencia de esta investigación se anexan las hojas de libreta con la simbología, tablas de verdad, sintaxis y diagramas de transistores de cada compuerta.

Posteriormente, se implementó un módulo combinacional en Verilog con dos entradas binarias (a y b) y siete salidas, una por cada operación lógica: out_and, out_or, out_not_a, out_nand, out_nor, out_xor y out_xnor. Al ser un circuito combinacional, las salidas dependen únicamente de los valores actuales de a y b, por lo que se utilizaron asignaciones continuas (assign) con operadores lógicos/bit a bit de Verilog: & para AND, | para OR, ~ para NOT, ^ para XOR y combinaciones con negación para NAND, NOR y XNOR. En esta implementación se calcula NOT solo para la entrada a, tal como se especifica en el módulo.

Para verificar el funcionamiento del diseño, se compararon los resultados esperados contra la teoría evaluando todas las combinaciones posibles de entrada (00, 01, 10, 11). La siguiente tabla resume los valores esperados (esto te sirve como verificación rápida en el reporte):

Tabla de verificación (a, b):

a b	AND	OR	NAND	NOR	XOR	XNOR	NOT(a)
00	0	0	1	1	0	1	1
01	0	1	1	0	1	0	1
10	0	1	1	0	1	0	0
11	1	1	0	0	0	1	0

Con esto se confirma que el módulo implementa correctamente las 7 compuertas lógicas solicitadas, por lo que puede considerarse como una base directa para las operaciones lógicas típicas de una ALU, y un ejemplo claro de cómo un HDL permite describir hardware que después puede simularse y, eventualmente, implementarse en un dispositivo reconfigurable como una FPGA.

CONCLUSIÓN

En esta actividad comprobé que las compuertas lógicas son la base real de las operaciones que ejecuta una ALU, porque a partir de reglas simples (tablas de verdad) se pueden construir funciones más complejas dentro de un procesador. También entendí mejor la diferencia entre programar “software” y describir “hardware”: en Verilog no se escribe una secuencia de pasos, sino relaciones lógicas que se cumplen al mismo tiempo, y eso facilita representar circuitos combinacionales de forma directa usando operadores como &, |, ~ y ^. Finalmente, al relacionar Verilog con FPGAs me queda claro que el diseño puede pasar de una descripción a una implementación real en un chip reconfigurable, lo cual hace el proceso de diseño y verificación más práctico y flexible en sistemas digitales.

REFERENCIAS EN FORMATO APA

Mano, M. M. R., & Ciletti, M. D. (2022). *Digital design: With an introduction to the Verilog HDL, VHDL, and SystemVerilog* (6th ed.). Pearson.

Brown, S., & Vranesic, Z. (2013). *Fundamentals of digital logic with Verilog design* (3rd ed.). McGraw-Hill Education.

Compton, K., & Hauck, S. (2002). Reconfigurable computing: A survey of systems and software. *ACM Computing Surveys*, 34(2), 171–210.

Institute of Electrical and Electronics Engineers. (2006). *IEEE Standard for Verilog Hardware Description Language (IEEE Std 1364-2005)*. IEEE.

Institute of Electrical and Electronics Engineers. (2018). *IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language (IEEE Std 1800-2017)*. IEEE.